РЕШЕН ИЗПИТ 1 — С ПЪЛНИ ОБЯСНЕНИЯ

Базови Програмни Езици — C (plain C)

Изпит 1: Аптека — Лекарства | 75 точки

Теми: struct + fscanf от текстов файл | модификация по условие | fwrite в бинарен файл | изтриване от масив

Изпитът покрива четири задачи — всяка с различна ключова техника.

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ НА ЗАДАЧАТА

Четири задачи с различни точки. Задача 1 чете от текстов файл в динамичен масив — шаблон, различен от предишните изпити. Задача 4 (изтриване) е нов шаблон, важен за Задача 4 на Изпит 3.

КОД / ИЗРАЗ	КАКВО ОЗНАЧАВА ЗА НАС
`наименование (низ до 30 символа)`	char name[31]; — 30+1 за нулевия терминатор
`дата на годност ММ.ГГГГ`	char expiry[8]; — 07.2025 = 7 символа + \0
`код (тринадесетцифрено цяло число)`	long long code; — int не побира 13 цифри!
`единична цена (реално число)`	float price; — при printf: %.2f
`налично количество (цяло число)`	int quantity;
`Прочетете данните от файла`	fscanf() в while цикъл — не fread!
`намалява с 20% цената`	price *= 0.80; (или price = price * 0.80)
`записва в offer.bin`	fwrite() с "wb" — бинарен запис
`изтрива лекарството от масива`	shift left шаблон: arr[i] = arr[i+1] в цикъл назад

СЕКЦИЯ 1 — ПЪЛЕН КОД (gcc -Wall -std=c99)

ВАЖНО: long long е задължително за 13-цифрен код — int и long не са достатъчно големи!

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char      name[31];     /* до 30 символа + '\0' */
    char      expiry[8];    /* ММ.ГГГГ = 7 символа + '\0' */
    long long code;         /* 13 цифри — ЗАДЪЛЖИТЕЛНО long long! */
    float     price;        /* единична цена */
    int       quantity;     /* налично количество */
} Medicine;

/* — ЗАДАЧА 1: четене от текстов файл — */
Medicine* loadFromFile(const char *fn, int *count) {
    FILE *f = fopen(fn, "r");
    if (!f) { *count = 0; return NULL; }

    Medicine *arr = NULL;
    *count = 0;
    Medicine tmp;
    /* Формат: name;expiry;code;price;quantity */
    while (fscanf(f, "%30[^;];%7[^;];%lld;%f;%d\n",
           tmp.name, tmp.expiry, &tmp.code,
           &tmp.price, &tmp.quantity) == 5) {
        (*count)++;
        arr = (Medicine*)realloc(arr, (*count) * sizeof(Medicine));
        if (!arr) { fclose(f); return NULL; }
        arr[(*count) - 1] = tmp;
    }
    fclose(f);
    return arr;
}

/* — ЗАДАЧА 2: намаляване с 20% — */
void discountExpiring(Medicine *arr, int count, const char *beforeDate) {
    int changed = 0;
    for (int i = 0; i < count; i++) {
        /* strcmp < 0: expiry идва ПРЕДИ beforeDate лексикографски */
        if (strcmp(arr[i].expiry, beforeDate) < 0) {
            float oldPrice = arr[i].price;
            arr[i].price *= 0.80f;  /* намаляване с 20% */
            printf("****************************************\n");
            printf("%s - %s - %.2flv - %.2flv\n",
                   arr[i].name, arr[i].expiry, oldPrice, arr[i].price);
            changed = 1;
        }
    }
    if (!changed) printf("Няма лекарства с изтичащ срок.\n");
}

/* — ЗАДАЧА 3: запис в бинарен файл — */
void saveAboveQty(Medicine *arr, int count, int minQty) {
    FILE *f = fopen("offer.bin", "wb");
    if (!f) { printf("Грешка при offer.bin!\n"); return; }
    for (int i = 0; i < count; i++) {
```

```c
        if (arr[i].quantity > minQty) {
            fwrite(&arr[i], sizeof(Medicine), 1, f);
        }
    }
    fclose(f);
}

/* — ЗАДАЧА 4: изтриване по код — */
void deleteByCode(Medicine *arr, int *count, long long code) {
    int idx = -1;
    for (int i = 0; i < *count; i++) {
        if (arr[i].code == code) { idx = i; break; }
    }
    if (idx == -1) { printf("Не е намерено лекарство с код %lld.\n", code); return; }

    /* Shift: преместваме всеки елемент ЕДИН НАЗАД */
    for (int i = idx; i < *count - 1; i++)
        arr[i] = arr[i + 1];  /* копира целия struct */
    (*count)--;
}

int main() {
    int count = 0;
    Medicine *arr = loadFromFile("medicines.txt", &count);
    if (!arr) { printf("Грешка при четене!\n"); return 1; }

    discountExpiring(arr, count, "01.2025");
    saveAboveQty(arr, count, 10);
    deleteByCode(arr, &count, 1234567890123LL);

    free(arr);
    return 0;
}
```

СЕКЦИЯ 2 — СТРУКТУРАТА И ЧЕТЕНЕ ОТ ТЕКСТОВ ФАЙЛ

Защо long long за 13-цифрен код?

int побира до ~2 милиарда (10 цифри). long long побира до ~9.2 квинтилиона (19 цифри). За 13-цифрен код е задължително long long — int или long ще препълнят!

```
[NE]  int code;        // max ~2,147,483,647 — 10 цифри. НЕ побира 13!
[OK]  long long code; // max ~9.2×10^18 — побира 13 цифри
```

Ред по ред: loadFromFile() — четене с fscanf

КОД	ОБЯСНЕНИЕ
`fopen(fn, "r")`	"r" = четене. Не "rb" — текстовият файл не е бинарен!
`Medicine tmp;`	Временна структура — четем в нея и после копираме в масива.
`fscanf(f, "%30[^;];%7[^;];%lld;%f;%d\n",`	%[^;] чете до разделителя ";". %lld е форматът за long long. %f за float. %d за int.
`tmp.name, tmp.expiry, &tmp.code,`	name и expiry са масиви — без &. code, price, quantity са скалари — С &!
`&tmp.price, &tmp.quantity) == 5`	fscanf връща броя успешно прочетени полета. == 5 = всички 5 са прочетени = успех. При EOF връща -1.
`arr[(*count) - 1] = tmp;`	Копираме целия struct с =. В С присвояването на struct копира всички полета.

Ред по ред: discountExpiring() — сравнение на дати като низове

Датите са в формат ММ.ГГГГ (напр. '07.2025'). strcmp() сравнява лексикографски — '07.2025' < '12.2025' работи, защото символите се сравняват по ASCII стойност отляво надясно.

КОД	ОБЯСНЕНИЕ
`strcmp(arr[i].expiry, beforeDate) < 0`	strcmp < 0 означава: expiry идва ПРЕДИ beforeDate. '07.2024' < '01.2025' = true = изтекъл срок.
`float oldPrice = arr[i].price;`	Запазваме старата цена ПРЕДИ промяната, за да я принтираме.
`arr[i].price *= 0.80f;`	*= 0.80 умножава по 0.80 = намалява с 20%. f суфиксът е за float константа.
`printf("%s - %s - %.2flv - %.2flv\n"`	Форматът е точно от условието: name - expiry - стара_цена - нова_цена.

Ред по ред: deleteByCode() — изтриване с shift

Изтриването от масив в C се прави чрез 'shift left' — преместваме всеки елемент след изтрития с един позиция назад, след което намаляваме брояча.

```
/* Преди изтриване (idx=1):  [A][B][C][D] */
/* i=1: arr[1] = arr[2] →    [A][C][C][D] */
/* i=2: arr[2] = arr[3] →    [A][C][D][D] */
/* (*count)-- →              [A][C][D]    */ (последният D е 'сляп' — count не го вижда)
```

КОД	ОБЯСНЕНИЕ
`for (int i = idx; i < *count - 1; i++)`	От позицията на изтрития до предпоследния елемент.
`arr[i] = arr[i + 1];`	Преместваме десния елемент наляво. arr[idx] се презаписва. arr[i+1] е целият struct.
`(*count)--;`	Намаляваме брояча. Последният елемент (дубликат) вече не е видим за програмата.

СЕКЦИЯ 3 — ЗАДАЧА 3: ЗАПИС В БИНАРЕН ФАЙЛ С УСЛОВИЕ

За разлика от Изпит 5 (където записвахме ЦЕЛИЯ масив наведнъж), тук записваме само тези елементи, които отговарят на условие — по един с fwrite в цикъл.

КОД	ОБЯСНЕНИЕ
`fopen("offer.bin", "wb")`	'wb' = write binary. ЗАДЪЛЖИТЕЛНО 'b'! Отваря нов файл — ако съществува, изтрива съдържанието.
`for (int i = 0; i < count; i++)`	Обхождаме ЦЕЛИЯ масив.
`if (arr[i].quantity > minQty)`	Проверяваме условието ПРЕДИ запис.
`fwrite(&arr[i], sizeof(Medicine), 1, f);`	Записваме ЕДИН елемент. &arr[i] = адрес на i-тия елемент. sizeof(Medicine) = точния размер.
`fclose(f);`	ЗАДЪЛЖИТЕЛНО — данните са в буфер до fclose()!

> **ВАЖНО:** Разлика: fwrite(arr, sizeof(Medicine), count, f) записва ВСИЧКИ. fwrite(&arr[i], sizeof(Medicine), 1, f) в цикъл с if записва само избраните.

Грешки характерни за Изпит 1

```
[NE]  int code;                          // препълване за 13-цифрен код!
[OK]  long long code;                    // задължително за 13 цифри

[NE]  scanf("%d", &tmp.code);            // %d е за int, не за long long!
[OK]  fscanf(f, "%lld", &tmp.code);      // %lld е форматът за long long

[NE]  arr[idx] = arr[idx+1]; count--;  // само 1 преместване — грешно!
[OK]  for (int i=idx; i<*count-1; i++) arr[i]=arr[i+1]; (*count)--;
```